ACE — Plataforma

Código fuente de la plataforma central y cliente reproductor de **ACE (Autonomous Content Engine)**: backend Laravel, panel administrativo territorial Filament y runtime del Player Electron para terminales Mini PC (Intel NUC).

La documentación de ingeniería, arquitectura, modelos de datos, decisiones y seguimiento del proyecto residen en el repositorio privado `ACE`. Este repositorio contiene exclusivamente la implementación de software.

Descripción del Producto

- **¿Qué es?:** ACE es una solución distribuida de cartelería digital inteligente (*Digital Signage*) para la programación, distribución y reproducción automatizada de contenidos multimedia institucional y publicitario.
- **¿A quién va dirigido?:** Administradores de infraestructura de TI, operadores de contenido y administradores de sede/locación que gestionan redes territoriales de pantallas físicas.
- **¿Qué problema resuelve?:** Busca evitar pantallas negras y cortes de contenido cuando falla la conexión a la red en recintos comerciales o educativos. Implementa una arquitectura *offline-first* con reproducción local en bucle continuo desde caché en disco, sincronización periódica tolerante a fallos, descargas segmentadas HTTP 206 Range, aislamiento territorial multi-tenant y validación criptográfica de integridad SHA-256.

Equipo de Desarrollo

Proyecto desarrollado para la asignatura de Proyecto de Título (PTY4614) de Duoc UC:

Integrante	Rol en el Proyecto
Tomás Escalante	Arquitectura, base de datos y backend
Matías Salas	Gestión, base de datos y vinculación con el cliente
Paulo Loyola	Frontend y documentación técnica

Metodología de Trabajo

- **Marco Ágil:** Scrum adaptado con sprints orientados a los hitos de evaluación de Capstone de Duoc UC.
- **Ingeniería Guiada por Calidad:** Desarrollo guiado por pruebas (TDD) e integración continua (CI) en GitHub Actions (Laravel Pint, Pest PHP y Node.js `node:test`).
- **Flujo de Git:** Ramas por frente de trabajo (`responsable/frente-resumen`), Pull Requests obligatorios con revisión cruzada de equipo y auditoría independiente (Heimdall).
- **Convención de Commits:** Conventional Commits 1.0.0 (`feat`, `fix`, `docs`, `test`, `refactor`).

Arquitectura y Stack Tecnológico

Backend y Plataforma Central

Componente	Versión / Tecnología	Propósito
PHP	8.3 o superior (Laravel Herd en Windows)	Runtime del servidor de aplicaciones
Laravel	13.x	Framework backend y API RESTful
Filament	5.x	Panel administrativo y gestión territorial

Componente	Versión / Tecnología	Propósito
Sanctum	4.x	Autenticación de las pantallas mediante tokens
MariaDB	11.4 LTS (Contenedor Docker)	Base de datos relacional multi-tenant
Pruebas	Pest 5.x	Pruebas unitarias y de integración del backend
Calidad	Laravel Pint	Formato de código con el preset de Laravel

Player Terminal (NUC / Dispositivos en Terreno)

Componente	Versión / Tecnología	Propósito
Electron	44.x	Runtime de escritorio para reproducción en pantalla completa
Node.js	22 LTS (v22.12+)	Proceso principal, sincronización periódica y servidor I/O
Motor de Render	Chromium	Reproducción a pantalla completa con transición continua entre medios
Almacenamiento	Sistema de archivos local	Caché local de medios identificados por SHA-256 (`media/<hash>`)
Pruebas	`node:test` nativo	Pruebas unitarias de resolución temporal y sincronización
Asistente	Batch (`Iniciar_Player.bat`)	Aprovisionamiento rápido e interactivo en terminales LAN

Requisitos

El entorno es mixto: PHP se ejecuta en la máquina y MariaDB corre en Docker.

1. Laravel Herd: instala PHP 8.4 y Composer.
2. Docker Desktop.
3. Node.js 22 LTS o superior.
4. FFmpeg, para leer la duración de los videos:

```
winget install Gyan.FFmpeg
```

El panel acepta videos de hasta 500 MB, pero PHP en Herd limita las subidas a 2 MB por defecto. En el `php.ini` de Herd (`php --ini` muestra su ubicación) fije `upload_max_filesize = 500M` y `post_max_size = 512M`, y reinicie Herd. `composer verificar` comprueba este límite.

Compruebe que la máquina cumple todo:

```
composer verificar
```

Instalación

```
git clone https://github.com/tomiloki/ace-plataforma.git
cd ace-plataforma
Copy-Item .env.example .env
docker compose up -d
composer setup
```

`docker compose up -d` levanta MariaDB con los datos de conexión definidos en `.env`. `composer setup` instala las dependencias, genera la clave de la aplicación, ejecuta las migraciones y compila los recursos del frontend.

Si el puerto 3306 ya está en uso en su máquina, cambie `DB_PORT` en `.env` antes de levantar MariaDB.

Cree un usuario para ingresar al panel:

```
php artisan make:filament-user
```

Ejecución

```
composer dev
```

- Aplicación: http://localhost:8000
- Panel administrativo: http://localhost:8000/admin

Para que un NUC de la red local alcance el backend durante las pruebas del Player:

```
php artisan serve --host=0.0.0.0
```

Además, el firewall de Windows debe permitir conexiones entrantes al puerto 8000.

Verificación

Antes de abrir una pull request:

```
composer test
vendor/bin/pint --test
```

`vendor/bin/pint` sin `--test` corrige el formato automáticamente.

Las pruebas se ejecutan contra MariaDB, en la base `ace_testing`, porque varias reglas del modelo viven en la base de datos. MariaDB debe estar levantado. La base de pruebas se crea automáticamente la primera vez que se inicializa el volumen de Docker. Si su volumen es anterior a este cambio, créela una vez con:

```
docker compose exec mariadb sh /docker-entrypoint-initdb.d/01-create-testing-database.sh
```

Estructura

La aplicación Laravel ocupa la raíz del repositorio. El subdirectorio `player/` contiene el cliente reproductor completo, estructurado de forma modular en JavaScript ESM puro (sin bundler):

- **Núcleo de resolución temporal (`player/src/core/`):** Algoritmo determinista de selección de playlists y cálculo de secuencias temporales.
- **Motor de sincronización y caché (`player/src/sync/`):** Heartbeat periódico, validación de manifest y descargas segmentadas HTTP 206 con verificación SHA-256 en disco.
- **Runtime de escritorio y renderizado (`player/src/electron/` y `player/src/renderer/`):** Proceso principal en Electron 44 y viewport en Chromium con soporte de conmutación gapless a pantalla completa.

Para ejecutar las pruebas unitarias del Player:

```
npm --prefix player test
```

El aprovisionamiento rápido y la ejecución del reproductor pueden realizarse interactivamente mediante `Iniciar_Player.bat` o a través de los scripts y opciones avanzadas detalladas en `player/README.md`.

Agentes de IA

El repositorio queda preparado para Claude Code, Codex y Antigravity sin configuración adicional:

- `AGENTS.md` contiene las únicas instrucciones que se cargan siempre. Debe mantenerse breve. `CLAUDE.md` lo importa.
- `.claude/skills` y `.agents/skills` contienen las skills, que se cargan solamente cuando la tarea las requiere: `filament-development`, `laravel-best-practices` y `testing-best-practices`.
- Las skills se generan con Laravel Boost. No se instalan sus guías permanentes, su servidor MCP ni sus reglas de proyecto.

La selección vive en `boost.json` (agentes y paquetes) y en `config/boost.php` (skills excluidas). Después de actualizar Laravel, Filament o Pest, regenere las skills y confirme el resultado:

```
php artisan boost:update --no-discover
```

No edite los archivos de las skills a mano: la regeneración los sobrescribe.

Colaboración

- `main` contiene la versión estable. El código no se sube directamente a `main`.
- Cada integrante trabaja en una rama por frente con el formato `responsable/frente-resumen`, por ejemplo `paulo/panel-filament`.
- La integración se realiza mediante pull request.
- Los commits siguen Conventional Commits.